

答辩 PPT 逐页讲解稿

项目：基于本地大模型与 RAG 技术的网站智能客服系统

说明：PPT 页面中不写建议时长；本讲解稿单独标注每页建议时间，用于 15 分钟左右答辩。讲解内容已同步新版 PPT 中的技术插图、页内补充信息和底部技术落点。

第 1 页：基于本地大模型与 RAG 技术的网站智能客服系统（0:00-0:45）

逐字稿：

各位老师好，我今天汇报的项目是《基于本地大模型与 RAG 技术的网站智能客服系统》。

这个项目面向的是 NXP AIoT Cloud、Cloud Lab、Edge AI 以及相关开发工具资料的问答场景。我们希望做出的不是一个简单的大模型聊天页面，而是一个可以在本地运行、能够检索本地知识库、能够展示答案来源和处理流程的网站智能客服系统。

本项目默认使用本地 Ollama 运行大语言模型，回答过程不依赖 OpenAI 或其他云端大模型 API。用户提问以后，系统先从本地知识库中找到相关资料，再把资料和问题一起交给本地模型生成回答。这样既能减少模型幻觉，也方便在答辩时展示系统的技术链路。

下面我会按照背景、需求、架构、知识库、RAG 流程、本地模型、前后端实现、测试结果和项目成果这几个部分进行汇报。

过渡：下面进入汇报内容。

第 2 页：汇报内容（0:45-1:20）

逐字稿：

这一页是本次汇报的目录，分为三个部分。

第一部分是项目介绍，主要回答为什么要做这个系统，以及需求目标是什么。这里重点是企业技术资料比较分散，普通 FAQ 很难覆盖自然语言提问，而直接让大模型回答又缺少来源约束。

第二部分是项目过程，我会重点讲总体架构、知识库构建、RAG 混合检索、本地模型部署、前端页面和后端 API。这里也是本项目和普通大模型套壳最大的区别：我们不是直接把问题交给模型，而是先检索资料、再构造 Prompt、再让本地模型生成回答。

第三部分是项目结果，包括测试验收、问题修复、最终成果和后续展望。整个汇报控制在 15 分钟左右。

过渡：下面先看项目背景。

第 3 页：项目背景：企业技术资料需要可解释问答（1:20-2:25）

逐字稿：

本项目的背景来自企业技术资料查询和课程演示需求。

以 NXP AIoT Cloud 和 Cloud Lab 为例，相关资料覆盖平台介绍、AI Hub、Model Zoo、LLM Edge Studio、VLM Edge Studio、Qwen 视频分析、VSS 视频检索、YOLOv8n 目标检测，以及 i.MX 和 MCU 应用范例等内容。这些资料本身很有价值，但是分布比较散，用户如果只靠网页目录或关键词搜索，理解成本比较高。

传统 FAQ 客服可以回答固定问题，但是面对自然语言变体和长尾问题时能力有限。另一方面，如果直接把问题交给通用大模型，模型可能并不了解我们项目中的本地资料，也可能把没有依据的内容说得很确定。

所以本项目采用 RAG，也就是检索增强生成。系统先从本地知识库找依据，再让本地大模型根据依据回答。PPT 右侧用四个小卡片概括了这个问题：资料分散、提问方式自然化、普通大模型缺少项目内依据、本地 RAG 能把答案和资料来源绑定。

这一页底部的技术落点也说明了本项目的核心原则：回答范围由本地知识库约束，答案必须能回到资料标题、来源和检索片段。

过渡：有了这个背景，下一页说明具体需求和目标。

第 4 页：需求与目标（2:25-3:30）

逐字稿：

这一页说明项目需求和目标。

从用户角度看，系统需要支持网页端提问，也要支持点击示例问题进行课堂演示。用户发送问题以后，页面不仅要显示回答，还要显示检索到的 Top-K 资料来源、相似度、处理步骤和耗时指标。

从后端角度看，系统需要提供健康检查、知识库统计、示例问题、索引重建和聊天问答接口。健康检查用于判断 Ollama、本地模型、知识库和向量库是否正常；索引重建用于把新加入的知识库资料重新切分并生成向量。

从部署角度看，本项目默认使用本地 Ollama，不依赖云端大模型 API。为了提高课堂演示稳定性，系统还设计了降级机制。比如 ChromaDB 不可用时退回本地 JSON 向量库，embedding 模型不可用时退回 TF-IDF，大模型不可用时可以使用 MOCK_LLM 展示完整流程。

这里还有一个项目要求是通过 Cody、Coze 等 AI 创作平台搭建 Agent 工作流，包括 RAG 数据库等。我们的实现方式是用 AI 创作平台辅助完成设计、代码、资料整理和答辩材料，但核心 Agent 流程、知识库、向量索引和接口都落在本地项目中，能够实际运行和复现。

所以这一页底部写的验收口径可以概括为六个字：能问、能检索、能生成、能追溯、能降级、能重建索引。

过渡：下面进入总体架构。

第 5 页：总体架构（3:30-4:45）

逐字稿：

系统总体架构分为四层，PPT 右侧是分层结构图。

第一层是前端交互层。前端使用原生 HTML、CSS 和 JavaScript，实现三栏页面，负责用户提问、系统状态展示、RAG 流程展示和检索来源展示。没有引入 React、Vue 或外部 CDN，主要是为了降低课堂演示环境依赖。

第二层是后端服务层。后端使用 FastAPI，负责健康检查、统计、索引重建和聊天接口，也负责把一次问答拆成多个可展示的流程状态。

第三层是 RAG 检索层。它包括文档加载、文本切分、embedding 生成、ChromaDB 向量检索、关键词补充召回、来源聚合和 Prompt 构造。这个层是系统区别于普通聊天页面的核心。

第四层是本地模型层。系统使用 Ollama 在本机运行 qwen2.5:7b 作为回答模型，并使用 nomic-embed-text 作为 embedding 模型。数据层则是 Markdown 知识库和本地向量索引，全部保存在项目目录内。

图下面的两条小卡片分别表示请求流和数据流。请求流是浏览器到 FastAPI，再到 RAG 和 Ollama；数据流是 Markdown 资料经过 chunks、embedding，最后进入 Vector DB。

过渡：接下来讲知识库是怎么构建的。

第 6 页：本地知识库构建（4:45-6:00）

逐字稿：

这一页介绍本地知识库构建，右侧这张图就是这次新增的知识库入库流程图。

RAG 系统的基础是知识库。如果资料质量不高，后面的检索和生成都会受到影响。因此我们没有把网页 HTML 直接整段复制进来，而是把资料整理成适合检索的 Markdown 文档。

当前知识库共 17 篇文档，覆盖 NXP AIoT Cloud、Cloud Lab、AI Hub、Model Zoo、模型转换与部署工具、LLM Edge Studio、VLM Edge Studio、Qwen2.5-VL、VSS 视频检索、YOLOv8n、多路视频流目标检测、i.MX 应用范例、MCU 应用范例和系统方案等主题。

图里的四个阶段对应官网资料、Markdown、Chunks 和 Vector DB。系统先保存结构化文档，再按固定参数切分成 134 个片段，并给每个片段保存标题、来源、类别等 metadata。向量化后优先写入 ChromaDB；如果 ChromaDB 不可用，则自动回退到 JSON 加 numpy 的本地检索。

这一页底部给出了入库参数：17 篇文档、134 个 chunks，CHUNK_SIZE 是 500，CHUNK_OVERLAP 是 80，TOP_K 默认是 3。chunk overlap 的作用是避免概念定义和应用场景刚好被切在两个片段中，影响检索效果。

过渡：知识库建好以后，下一步就是 RAG 检索和生成流程。

第 7 页：RAG 流程与混合检索（6:00-7:35）

逐字稿：

这一页是本项目最核心的 RAG 流程，右侧是新增的 RAG workflow 图。

用户在网页输入问题后，前端调用 POST /api/chat。后端首先记录流程状态，然后判断问题类型。如果用户问的是“你的资料数是多少”或者“当前知识库有哪些内容”，这类问题属于系统运行上下文问题，后端会直接根据实时统计回答，不会去知识库里硬找来源。

如果用户问的是具体技术问题，比如“介绍一下 RAG 智能客服”或者“NXP Cloud Lab 系统方案有哪些”，系统会进入普通 RAG 检索流程。首先把问题转换为 embedding 向量，然后在 ChromaDB 中做向量相似度检索。

在实际测试中，我们发现纯向量检索对短中文问题和专有名词有时不够稳定。例如“介绍一下 RAG 智能客服”这样的问法，向量检索可能会把泛化资料排在前面。因此我们加入了关键词、中文双字词和标题覆盖率评分，把向量召回和词项召回融合排序。

图下面的两个小卡片概括了这个设计：检索增强是先找资料、再生成答案；可追溯是标题、来源和相似度同步返回。最后系统按文档聚合来源，构造 Prompt，调用本地大模型生成回答。如果检索资料与问题无关，Prompt 会要求模型明确说明当前知识库没有明确依据，并且后端会清空无关 sources。

这一页底部的召回策略可以总结为：向量相似度加关键词和标题加权，低相关结果不进入最终来源。

过渡：下面看本地模型和降级机制。

第 8 页：本地模型与降级机制（7:35-8:45）

逐字稿：

这一页介绍本地模型和降级机制，右侧图展示的是本地机器、模型服务、向量库和备用路径。

回答模型默认是 qwen2.5:7b。这个模型规模相对适中，在普通实验室电脑上有机会运行，适合作为课程 Demo 的本地大模型。embedding 模型默认是 nomic-embed-text，用来把用户问题和知识库片段转换成向量。

向量库默认使用 ChromaDB，它可以把文档片段、metadata 和向量持久化保存。用户提问时，系统把问题向量和知识库向量进行相似度检索，找到最相关的资料片段。

为了保证演示稳定性，系统设计了多级降级。ChromaDB 不可用时，系统使用 SimpleVectorStore，把 chunk 和向量保存到本地 JSON 文件，并用 numpy 计算余弦相似度。Ollama embedding 不可用时，系统可以退回 TF-IDF。大模型生成不可用时，系统可以打开 MOCK_LLM，用本地 mock 回答展示前端和检索流程。

这些降级机制不是为了替代正式方案，而是为了避免课堂现场因为某个依赖出问题导致整个系统无法展示。底部的技术落点也说明了这一点：qwen2.5:7b 负责生成，nomic-embed-text 负责向量化，异常时保持可演示闭环。

过渡：下面看前端演示页面。

第 9 页：前端演示页面（8:45-9:45）

逐字稿：

这一页展示前端演示页面的设计。

前端采用三栏布局。左侧是系统状态区，可以看到 Ollama 是否连接、当前 LLM 模型、embedding 模型、知识库文档数、chunk 数和向量库类型，还可以点击按钮重建知识库索引。

中间是聊天区，包括用户和 AI 的对话消息、示例问题、Top-K 控制和底部输入框。示例问题覆盖系统架构、RAG 流程、本地部署优势、LLM Edge Studio、Ara240 DNPU 等答辩常见问题。

右侧是 RAG 流程和检索来源。流程分为六步：接收用户问题、问题向量化、知识库检索、Prompt 构造、本地大模型生成、返回答案与来源。每一步都会显示状态和说明。

这页也回应了之前测试中发现的前端问题。长问题、示例问题和回答区域都做了换行约束，避免投屏时超出屏幕。页面不依赖 React、Vue 或 CDN，后端启动后浏览器即可访问。

过渡：前端之外，下面看后端 API 和代码模块。

第 10 页：后端 API 与代码模块 (9:45-10:45)

逐字稿：

这一页介绍后端 API 和代码模块。

后端主要有几个接口。GET /api/health 用于返回系统状态，包括 Ollama 是否连接、模型名、知识库是否加载、文档数、chunk 数和向量库类型。GET /api/stats 返回知识库统计和分类分布。GET /api/sample-questions 返回课堂演示问题。POST /api/rebuild-index 用于清理旧索引并重新构建本地向量库。POST /api/chat 是核心问答接口，负责执行检索、Prompt 构造、本地模型回答和来源返回。

代码结构也按职责拆分。app/main.py 负责 FastAPI 路由和整体流程；app/ollama_client.py 封装 Ollama 的 health、chat 和 embed 调用；app/rag/document_loader.py 负责读取 Markdown、TXT 和 JSON；chunker.py 负责切分文本；embeddings.py 负责 Ollama embedding 和 TF-IDF fallback；vector_store.py 负责 ChromaDB 和 SimpleVectorStore；retriever.py 负责混合检索；prompt_builder.py 负责构造 Prompt。

接口响应中包含 answer、sources、metrics、steps 等字段，前端可以直接根据这些字段渲染回答、来源和流程状态。这种模块化结构也便于测试和答辩说明。

过渡：下面进入测试与验收结果。

第 11 页：测试与验收结果 (10:45-12:00)

逐字稿：

这一页是测试与验收结果，右侧新增的图表示自动化测试、接口检查、验收清单和运行指标。

第一类测试是单元测试。pytest 当前 8 项测试通过，覆盖文本切分、Prompt 构造、文档加载、schema 序列化、向量检索、运行上下文问题识别，以及关键词召回优先级。

第二类测试是 smoke_test。它会在 MOCK_LLM 和 TF-IDF 模式下检查首页、健康检查、统计接口、示例问题、索引重建和聊天接口，保证没有完整 Ollama 环境时也能验证基础流程。

第三类测试是 acceptance_check。它覆盖任务书要求的目录结构、配置文件、README 和文档、前端关键元素、无 Ollama 降级、重建索引、MOCK 聊天和无来源提示。

第四类是真实接口批测。我们测试了资料数量、知识库范围、系统能力、RAG 流程、NXP 技术问题，以及天气和股票这类超出知识库的问题。验收重点不是只看自然语言回答，还要看来源、流程、耗时和降级状态是否正确。

底部的测试组合可以概括为：单元测试、smoke test、acceptance check 和真实问答批测。

过渡：测试过程中发现的问题，下一页说明怎么修复。

第 12 页：问题修复与迭代优化（12:00-13:05）

逐字稿：

项目迭代中遇到了几个典型问题，这些问题也体现了工程调试过程。

第一个问题是短问法召回不稳定。比如用户问“介绍一下 RAG 智能客服”，早期纯向量检索有时会把泛化资料排在前面。解决方法是加入混合召回，增加关键词和标题覆盖率评分，让相关文档稳定排到前面。

第二个问题是资料数量问题回答死板。比如用户问“你的资料数是多少”，这不是知识库资料问题，而是系统运行状态问题。我们没有写死固定答案模板，而是加入运行上下文问题识别，让后端实时注入文档数、chunk 数、向量库类型和资料范围。

第三个问题是来源重复。同一个文档的多个 chunk 可能同时进入 Top-K，导致前端显示多个相同标题。我们后来按 doc_id 聚合来源，每篇文档最多合并两个片段，既保留信息量，也减少重复。

第四个问题是索引重建异常。simple 和 Chroma 切换后可能出现 readonly database。我们修改了构建索引逻辑，在构建 Chroma 前清理旧索引文件，保证前端按钮重建索引时能返回正常 JSON。

最后一个是前端超屏。长示例问题、长回答和底部输入区域都补齐了 word-wrap、overflow-wrap 和响应式约束，保证投屏时页面不会横向撑开。

过渡：下面总结项目成果和价值。

第 13 页：项目成果与价值（13:05-14:20）

逐字稿：

这一页总结项目成果和价值。

最终我们完成了一个完整可运行的 FastAPI + Ollama + RAG + 原生前端项目。系统具备网页问答、知识库检索、本地模型生成、来源展示、流程可视化、健康检查、索引重建和异常降级能力。

知识库方面，当前有 17 篇本地文档，134 个 chunks，覆盖 NXP AIoT Cloud、Cloud Lab、AI Hub、Model Zoo、LLM/VLM Edge Studio、Qwen/VSS、YOLOv8n、i.MX/MCU 应用范例和系统方案。

工程交付方面，项目包含代码仓库、知识库、向量索引、运行脚本、README、API 文档、设计文档、测试脚本、报告、PPT 和逐页讲解稿。测试方面，pytest、smoke_test 和 acceptance_check 都已经通过。

这个项目的价值在于，它不是简单展示大模型生成能力，而是展示企业资料如何被整理、检索、引用和解释。对于课程项目来说，它能清楚展示完整工程闭环；对于企业场景来说，它可以作为内部资料客服或技术支持助手的原型。

右侧四个指标卡片也对应最终成果：17 篇知识库文档、134 个 chunks、4 个核心接口和 8 项单元测试。

过渡：最后做总结与展望。

第 14 页：总结与展望 (14:20-15:00)

逐字稿：

最后做一个总结。

本项目验证了本地 RAG 智能客服的技术可行性。系统从资料入库开始，经过文本切分、向量索引、混合检索、Prompt 构造、本地模型生成，最终在网页上展示回答和来源，形成了完整闭环。

相比普通大模型聊天页面，本项目的核心特点是本地化、可解释和可降级。本地化体现在默认使用 Ollama 和本地知识库；可解释体现在回答带有参考来源和相似度；可降级体现在 ChromaDB、embedding 和 LLM 不可用时仍有备用路径。

后续如果继续扩展，可以加入知识库上传、增量索引、多轮会话、用户反馈、重排序模型和真实边缘设备资料接入。这样系统可以从课堂 Demo 进一步发展为企业内部技术资料问答平台。

我的汇报到这里结束，感谢各位老师，欢迎批评指正。